Linux Storage Deep Dive

Mount Points · Swap Storage · LVM (Logical Volume Manager)

A complete RHCSA-focused reference and hands-on practice workbook covering disk partitioning, filesystem creation, mounting (manual and persistent), swap space management, and the full LVM stack (PV → VG → LV) with resizing. Every command includes an explanation, the exact configuration file it touches, and where that file lives on disk.

Topic	What you'll learn
`Storage Stack Basics`	Block devices, partitions, /dev naming, lsblk, blkid, fdisk/parted
`Filesystems`	mkfs, XFS vs ext4, UUID/labels, filesystem metadata
`Mounting`	mount, umount, /etc/fstab, systemd .mount units, mount options
`Swap`	mkswap, swapon/swapoff, /proc/swaps, swappiness, swap files
`LVM`	pvcreate, vgcreate, lvcreate, extend/reduce, /etc/lvm/, thin provisioning
`Practice Labs`	6 hands-on scenarios with step-by-step solutions

Table of Contents

1. The Linux Storage Stack — Big Picture

Before touching commands, understand the **layers** data passes through. Every layer wraps the one below it:

- **Physical Disk** — the raw block device, e.g. `/dev/sdb` or `/dev/nvme0n1`
- **Partition** — a slice of the disk, e.g. `/dev/sdb1` (optional if using whole-disk LVM)
- **LVM (optional layer)** — Physical Volume → Volume Group → Logical Volume, adds flexibility to resize later
- **Filesystem** — XFS, ext4, swap — formats the block device/LV so the kernel can store files
- **Mount Point** — a directory in the tree (e.g. `/data`) where the filesystem becomes accessible

NOTE: Swap does NOT need a mount point or filesystem in the normal sense — it is formatted with `mkswap` (a special swap signature) and activated with `swapon` instead of `mount`.

Why LVM sits between the partition and the filesystem

A plain partition has a fixed size baked into the partition table — growing it later is painful and often requires unmounting or rebooting. LVM adds a virtualization layer: it pools one or more partitions/disks (Physical Volumes) into a Volume Group, then carves out Logical Volumes of any size from that pool. You can grow, shrink, or move an LV without touching the underlying partition table, which is why RHCSA and real production systems use LVM almost everywhere except `/boot`.

---

2. Viewing & Identifying Disks

lsblk — list block devices as a tree

```
lsblk
lsblk -f # show filesystem type + UUID + mountpoint
lsblk -p # show full /dev/ paths
```

Shows every block device (disks, partitions, LVs, swap) in a parent→child tree. `-f` adds FSTYPE, UUID, and where each is mounted — the single most useful command for orienting yourself on an unfamiliar system.

blkid — show UUID, LABEL, and TYPE of a device

```
blkid
blkid /dev/sdb1
```

Reads filesystem metadata (superblock) to report the UUID, LABEL, and TYPE (xfs/ext4/swap). This is exactly the UUID you paste into `/etc/fstab`.

fdisk -l / parted -l — list partition tables

```
fdisk -l
fdisk -l /dev/sdb
parted -l
```

Lists disks with their partition table type (MBR/msdos vs GPT), size, and existing partitions.

df -h and du -sh — usage, not identity

```
df -hT # mounted filesystems + type + usage
du -sh /var/log # size of a directory tree
```

`df` reports space on already-mounted filesystems; `du` reports how much a directory actually consumes. They can disagree (e.g. a deleted-but-open file still counted by df).

3. Partitioning a Disk

Skip this step entirely if you plan to hand the whole raw disk to LVM as a Physical Volume — partitioning is only required for MBR/BIOS boot limits, dual-booting, or when policy demands it. RHCSA exams commonly still test it, so know both tools.

fdisk — classic, interactive, MBR-friendly (also supports GPT)

```
fdisk /dev/sdb
n # new partition
p # primary
1 # partition number
<Enter> # accept default first sector
+2G # size
t # change partition type
8e # Linux LVM (or 82 for swap)
w # write and exit
```

Everything happens inside an interactive prompt. Nothing is written to disk until you press `w` — `q` aborts safely.

parted — scriptable, GPT-first, works on disks > 2TB

```
parted /dev/sdb
mklabel gpt
mkpart primary 0% 2GiB
print
quit

# One-liner, non-interactive:
parted -s /dev/sdb mklabel gpt mkpart primary 0% 2GiB
```

Tell the kernel about the new partition table

```
partprobe /dev/sdb
# or
partprobe
```

After editing a partition table, the running kernel may still hold the old layout in memory. `partprobe` re-reads it without a reboot.

TIP: Partition type codes matter for LVM: `8e` (fdisk) / `lvm` flag (parted) marks a partition as "Linux LVM" so tools like `pvcreate` recognize intent, though it's advisory, not enforced by the kernel.

4. Creating Filesystems (mkfs)

A partition or LV is just raw blocks until you write a filesystem structure onto it. RHEL 9 defaults to **XFS** for most mounts; **ext4** remains common and is easier to shrink.

Command	Filesystem	Notes
`mkfs.xfs /dev/sdb1`	XFS	RHEL 9 default. Cannot shrink (only grow) — plan sizing carefully.
`mkfs.ext4 /dev/sdb1`	ext4	Supports both grow and shrink. Good for volumes you may need to reduce.
`mkfs.vfat /dev/sdb1`	FAT32	Used for EFI system partitions, USB interchange.
`mkswap /dev/sdb2`	swap	Not a real filesystem — writes a swap signature (see Section 6).

Labeling a filesystem (optional, but handy for fstab)

```
xfs_admin -L mydata /dev/sdb1 # XFS: set label (unmounted)
e2label /dev/sdb1 mydata # ext4: set label
```

Checking a filesystem for errors

```
xfs_repair /dev/sdb1 # XFS — must be UNmounted
fsck.ext4 -f /dev/sdb1 # ext4 — must be UNmounted
```

NOTE: Never run fsck/xfs_repair on a mounted filesystem — always umount first, or boot into rescue/single-user mode for root.

5. Mounting Storage — Manual & Persistent

5.1 Manual (temporary) mount

```
mkdir -p /data
mount /dev/sdb1 /data

# Mount by UUID instead of device name (more stable):
mount UUID="1a2b3c4d-..." /data

# Specify filesystem type & options explicitly:
mount -t xfs -o defaults,noatime /dev/sdb1 /data
```

A manual mount is **NOT persistent** — it disappears on reboot. It's fine for testing, but production mounts must go into `/etc/fstab` (Section 5.3).

5.2 Unmounting

```
umount /data
umount /dev/sdb1

# If "target is busy":
lsof +D /data # see what's using it
fuser -vm /data # alternative
umount -l /data # lazy unmount (last resort)
```

5.3 Persistent mounts — /etc/fstab

`/etc/fstab`

The kernel/systemd reads this file at every boot to mount filesystems automatically. Each line has 6 whitespace-separated fields:

Field	Meaning	Example
`1. Device`	UUID, LABEL, or /dev path (UUID strongly preferred)	UUID=1a2b3c4d-....
`2. Mount point`	Directory the filesystem attaches to	/data
`3. Type`	Filesystem type	xfs
`4. Options`	Comma-separated mount options	defaults
`5. Dump`	Legacy backup flag (0 = ignore, almost always 0)	0
`6. Pass`	fsck order at boot (0=skip, 1=root, 2=others)	0 for xfs* / 2 for ext4

* XFS ships its own consistency checking at mount time and is generally left at fsck pass 0.

Example /etc/fstab line

```
UUID=1a2b3c4d-5e6f-7890-abcd-1234567890ab /data xfs defaults 0 0
```

Always test before rebooting!

```
mount -a # mount everything in fstab (dry-run-ish sanity check)
# If mount -a returns to the prompt with no errors, fstab is syntactically valid.
findmnt /data # confirm it actually mounted where expected
```

WARNING: A typo in /etc/fstab can drop the system into **emergency mode** at next boot. Always run `mount -a` after editing, and keep a root/rescue shell handy while practicing.

5.4 Systemd .mount units (alternative to fstab)

/etc/systemd/system/data.mount

systemd can generate a native .mount unit from fstab automatically, or you can write one directly. The unit file name must match the mount path with slashes replaced by dashes (e.g. `/data` → `data.mount`; `/mnt/data` → `mnt-data.mount`).

```
[Unit]
Description=Data mount

[Mount]
What=/dev/sdb1
Where=/data
Type=xfs
Options=defaults

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl start data.mount
systemctl enable data.mount
```

5.5 Useful mount options

Option	Effect
`defaults`	rw, suid, dev, exec, auto, nouser, async — the sane baseline
`noatime`	Don't update file access-time on every read — improves performance
`ro`	Mount read-only
`nosuid`	Ignore SUID/SGID bits — security hardening
`noexec`	Prevent executing binaries from this mount
`nodev`	Don't interpret device files on this mount
`_netdev`	Wait for network before mounting (NFS, iSCSI)

5.6 Inspecting current mounts

```
mount | grep sdb1
cat /proc/mounts
findmnt # tree view of everything mounted
findmnt /data # details for one mount point
```

/proc/mounts (live kernel view — always accurate)

/etc/mtab (usually a symlink to /proc/self/mounts on modern systems)

6. Swap Storage

Swap is disk space the kernel uses as an overflow for RAM when physical memory is under pressure. It is slower than RAM but prevents out-of-memory kills. It can live on a dedicated **partition** or a **swap file**.

6.1 Creating swap on a partition

```
fdisk /dev/sdb # create a partition, set type 82 (swap) or 8200 (GPT)
mkswap /dev/sdb2 # write the swap signature
swapon /dev/sdb2 # activate it immediately
```

6.2 Creating a swap FILE (no free disk/partition needed)

```
# Allocate a 1GB file (fallocate is fast; dd is the portable fallback)
fallocate -l 1G /swapfile
# or: dd if=/dev/zero of=/swapfile bs=1M count=1024

chmod 600 /swapfile # must not be world-readable
mkswap /swapfile
swapon /swapfile
```

6.3 Making swap persistent

/etc/fstab

```
UUID=aa11bb22-... none swap sw 0 0
# or for a swap file:
/swapfile none swap sw 0 0
```

Notice fields 2 and 3: mount point is literally `none` (swap has no directory), and type is `swap`.

6.4 Managing active swap

```
swapon --show # list active swap devices/files with usage
swapon -a # activate everything listed in fstab
free -h # RAM + swap totals, human readable
swapoff /dev/sdb2 # deactivate (kernel migrates pages back to RAM)
swapoff -a # deactivate ALL swap
```

/proc/swaps (live kernel view of active swap areas)

6.5 Swappiness — how eagerly the kernel swaps

```
cat /proc/sys/vm/swappiness # current value, 0-100 (default 60)
sysctl vm.swappiness=10 # change now (not persistent)

# Persist across reboots:
echo 'vm.swappiness=10' >> /etc/sysctl.d/99-swappiness.conf
sysctl --system # reload all sysctl config
```

/etc/sysctl.d/99-swappiness.conf (or /etc/sysctl.conf)

Lower swappiness (e.g. 10) tells the kernel to prefer keeping data in RAM and only swap under real pressure — common on servers with plenty of RAM. Higher values swap more aggressively, freeing RAM for page cache sooner.

EXAM TIP: RHCSA sizing rule of thumb historically taught: swap = RAM size for RAM ≤ 2GB; swap = 0.5× RAM for 2–8GB; smaller ratios beyond that. Modern systems with lots of RAM often use much less proportionally, or rely on zswap/hibernation needs — always check the actual exam/task requirement.

7. LVM — Physical Volumes, Volume Groups, Logical Volumes

LVM has exactly three layers. Data flows: **disk/partition** → **PV** → **VG** → **LV** → **filesystem** → **mount**.

Layer	Command to create	Analogy
`Physical Volume (PV)`	pvcreate	A raw ingredient — a disk or partition registered for LVM use
`Volume Group (VG)`	vgcreate	A shared pool combining one or more PVs into one storage pot
`Logical Volume (LV)`	lvcreate	A slice cut from the VG pot — this is what you format & mount

7.1 Physical Volumes (PV)

```
pvcreate /dev/sdb1 /dev/sdc1 # mark partitions (or whole disks) as PVs
pvdisplay # detailed PV info
pvs # quick summary table
pvremove /dev/sdb1 # undo (must remove from VG first)
```

`pvcreate` writes an LVM metadata header near the start of the device. This is why PVs can be whole raw disks — no partition table required if you prefer.

7.2 Volume Groups (VG)

```
vgcreate vg_data /dev/sdb1 /dev/sdc1
vgdisplay vg_data
vgs
vgextend vg_data /dev/sdd1 # add another PV into the pool later
vgreduce vg_data /dev/sdc1 # remove a PV (must be empty first)
```

A VG pools the capacity of all its PVs into extents (default 4MiB chunks, see PE Size in vgdisplay). LVs are then allocated from this shared extent pool, which is why one VG can back many LVs of any size that fits.

7.3 Logical Volumes (LV)

```
lvcreate -n lv_data -L 5G vg_data # fixed size, 5 GiB
lvcreate -n lv_data -l 100%FREE vg_data # use all remaining free extents
lvdisplay /dev/vg_data/lv_data
lvs
```

The resulting device node is `/dev/vg_data/lv_data` (a symlink under `/dev/mapper/vg_data-lv_data`). Format and mount it exactly like a normal partition:

```
mkfs.xfs /dev/vg_data/lv_data
mkdir -p /data
mount /dev/vg_data/lv_data /data

# Persistent line for /etc/fstab:
/dev/vg_data/lv_data /data xfs defaults 0 0
```

7.4 Where LVM metadata actually lives

Path	What it holds
`/etc/lvm/lvm.conf`	Global LVM daemon/tooling configuration
`/etc/lvm/backup/`	Automatic text backup of VG metadata after every change
`/etc/lvm/archive/`	Historical metadata archive (for vgcfgrestore rollback)
`/dev/mapper/vgname-lvname`	Actual device-mapper node created for each LV
`/dev/vgname/lvname`	Convenience symlink to the device-mapper node

The PV header itself (UUID, VG membership) is stored directly on the device — `pvdisplay -v` or `pvck` can inspect it.

8. Resizing LVM (Extend & Reduce)

This is the single biggest reason LVM exists. Two steps are always required: (1) resize the **logical volume** block device, then (2) resize the **filesystem** living inside it — they are independent operations.

8.1 Extending (growing) — always safe, do it live

```
# Step 1: grow the LV (add 2GB)
lvextend -L +2G /dev/vg_data/lv_data
# or grow to use ALL remaining free space in the VG:
lvextend -l +100%FREE /dev/vg_data/lv_data

# Step 2: grow the filesystem to fill the new space
xfs_growfs /data # XFS — needs the MOUNT POINT, not device
# resize2fs /dev/vg_data/lv_data # ext4 — needs the DEVICE
```

SHORTCUT: One-liner shortcut: `lvextend -r -L +2G /dev/vg_data/lv_data` — the `-r` flag auto-resizes the filesystem in the same command (calls xfs_growfs or resize2fs internally based on detected type).

8.2 Reducing (shrinking) — ext4 only, XFS cannot shrink

XFS has **no shrink capability** — the only way to "shrink" an XFS LV is to back up the data, recreate a smaller LV/filesystem, and restore. ext4 supports true shrinking, but the filesystem **must be unmounted** first and shrunk before the LV:

```
umount /data
e2fsck -f /dev/vg_data/lv_data # mandatory consistency check first
resize2fs /dev/vg_data/lv_data 3G # shrink filesystem to 3G FIRST
lvreduce -L 3G /dev/vg_data/lv_data # THEN shrink the LV to match
mount /data
```

CRITICAL RULE: Order matters and is opposite of extending: when **growing**, resize the LV first, then the filesystem. When **shrinking**, resize the filesystem first (to something ≤ target), then the LV. Shrinking the LV below the filesystem's current size destroys data.

8.3 Removing LVM objects (cleanup order)

```
umount /data
lvremove /dev/vg_data/lv_data
vgremove vg_data
pvremove /dev/sdb1
```

Cleanup always goes top-down: unmount → remove LV → remove VG → remove PV. Skipping a step (e.g. vgremove while an LV still exists) will simply be refused by the tool.

9. Key Configuration & State Files — Master Table

File / Path	Purpose	Category
`/etc/fstab`	Persistent mounts (filesystems AND swap) read at boot	Mount + Swap
`/proc/mounts`	Live, always-accurate kernel view of current mounts	Mount
`/etc/mtab`	Legacy mount table, usually symlinked to /proc/self/mounts	Mount
`/etc/systemd/system/*.mount`	Native systemd mount unit files (fstab alternative)	Mount
`/proc/swaps`	Live view of currently active swap areas	Swap
`/etc/sysctl.d/99-swappiness.conf`	Persisted vm.swappiness and other kernel tuning	Swap
`/proc/sys/vm/swappiness`	Live current swappiness value (runtime, not persistent)	Swap
`/etc/lvm/lvm.conf`	Global LVM tool/daemon configuration	LVM
`/etc/lvm/backup/`	Auto text-backup of VG metadata after every change	LVM
`/etc/lvm/archive/`	Historical metadata snapshots for rollback (vgcfgrestore)	LVM
`/dev/mapper/vg-lv`	Device-mapper node for each Logical Volume	LVM
`/dev/vgname/lvname`	Convenience symlink to the device-mapper node	LVM
`/etc/blkid.conf / blkid cache`	Cached UUID/LABEL/TYPE lookups used by blkid	Filesystem
`/sys/block/`	Kernel sysfs view of all block devices (low-level)	Storage

10. Command Cheat-Sheet — Everything in One Table

Command	Category	What it does
`lsblk -f`	Discover	Tree view of block devices with FS type/UUID/mountpoint
`blkid`	Discover	Show UUID/LABEL/TYPE for devices
`fdisk -l / parted -l`	Discover	List partition tables
`fdisk /dev/sdX`	Partition	Interactive MBR/GPT partition editor
`parted /dev/sdX`	Partition	Scriptable partition editor, GPT-first
`partprobe`	Partition	Re-read partition table into running kernel
`mkfs.xfs / mkfs.ext4`	Filesystem	Create a filesystem on a device
`xfs_admin -L / e2label`	Filesystem	Set a filesystem label
`xfs_repair / fsck`	Filesystem	Check/repair an unmounted filesystem
`mount / umount`	Mount	Attach / detach a filesystem manually
`mount -a`	Mount	Mount everything listed in /etc/fstab
`findmnt`	Mount	Tree/detail view of active mounts
`mkswap`	Swap	Write a swap signature to a device/file
`swapon / swapoff`	Swap	Activate / deactivate swap
`swapon --show / free -h`	Swap	View active swap / memory summary
`pvcreate / pvs / pvdisplay`	LVM	Create & inspect Physical Volumes
`vgcreate / vgs / vgdisplay`	LVM	Create & inspect Volume Groups
`vgextend / vgreduce`	LVM	Add/remove PVs from a VG
`lvcreate / lvs / lvdisplay`	LVM	Create & inspect Logical Volumes
`lvextend -r -L +size`	LVM	Grow LV and filesystem together
`resize2fs / xfs_growfs`	LVM	Grow filesystem to match LV size
`lvreduce`	LVM	Shrink LV (ext4 filesystem must shrink first)
`lvremove / vgremove / pvremove`	LVM	Tear down LVM objects (top-down order)

Lab 1: Add a New Disk & Mount It Permanently

Goal: Attach a new 2GB disk, partition it, format as XFS, and mount it persistently at /data.

Steps

```
lsblk # confirm new disk shows up, e.g. /dev/sdb
fdisk /dev/sdb # n, p, 1, <enter>, +2G, w
partprobe
mkfs.xfs /dev/sdb1
blkid /dev/sdb1 # copy the UUID
mkdir -p /data
vi /etc/fstab # append the UUID line (Section 5.3)
mount -a
df -hT /data # verify
```

Self-check: *Does* `findmnt /data` *show the right UUID and fstype? Would it survive* `reboot`*?*

Lab 2: Create & Enable Swap

Goal: Add 512MB of swap using a swap file (no spare partition available) and make it persistent.

Steps

```
free -h # baseline
fallocate -l 512M /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
free -h # confirm swap increased
echo '/swapfile none swap sw 0 0' >> /etc/fstab
swapoff /swapfile && swapon -a # test fstab line works standalone
swapon --show
```

Self-check: *Does* `cat /proc/swaps` *show /swapfile? What happens to* `free -h` *output if you swapoff it?*

Lab 3: Build an LVM Stack from Scratch

Goal: Turn two new disks into one Volume Group, carve a 3GB Logical Volume, format it ext4, and mount at /lvdata.

Steps

```
lsblk # identify /dev/sdc, /dev/sdd (or similar)
pvcreate /dev/sdc /dev/sdd
pvs
vgcreate vg_lab /dev/sdc /dev/sdd
vgs
lvcreate -n lv_lab -L 3G vg_lab
mkfs.ext4 /dev/vg_lab/lv_lab
mkdir -p /lvdata
echo '/dev/vg_lab/lv_lab /lvdata ext4 defaults 0 2' >> /etc/fstab
mount -a
df -hT /lvdata
```

Self-check: *What does* `vgs` *report for VFree after creating a 3G LV from two disks totaling more than 3G? Where did the leftover space go?*

Lab 4: Extend a Logical Volume Online

Goal: Grow lv_lab from Lab 3 by 1GB using leftover VG space, live, with the filesystem still mounted.

Steps

```
vgs vg_lab # confirm free extents exist
lvextend -r -L +1G /dev/vg_lab/lv_lab # extends LV + resize2fs in one shot
lvs
df -hT /lvdata # verify new size, no downtime
```

Self-check: *Try the two-step version instead — `lvextend -L +1G` then separately `resize2fs` — and confirm you get the same result as the -r shortcut.*

Lab 5: Shrink an ext4 Logical Volume

Goal: Reduce lv_lab down to 2GB safely (ext4 supports shrink; XFS does not).

Steps

```
umount /lvdata
e2fsck -f /dev/vg_lab/lv_lab
resize2fs /dev/vg_lab/lv_lab 2G # shrink FS first
lvreduce -L 2G /dev/vg_lab/lv_lab # then shrink LV to match
mount /lvdata
df -hT /lvdata
```

WHY THIS MATTERS: If you ran `lvreduce` BEFORE `resize2fs` here, you would truncate the filesystem's backing storage while the filesystem still believes it owns the larger size — guaranteed data corruption. This lab exists specifically to drill the correct order.

Lab 6: Troubleshoot a Broken /etc/fstab (Boot Failure Recovery)

Goal: Simulate a bad fstab entry, learn to recognize the emergency-mode symptom, and repair it.

Steps — break it (in a lab/VM only!)

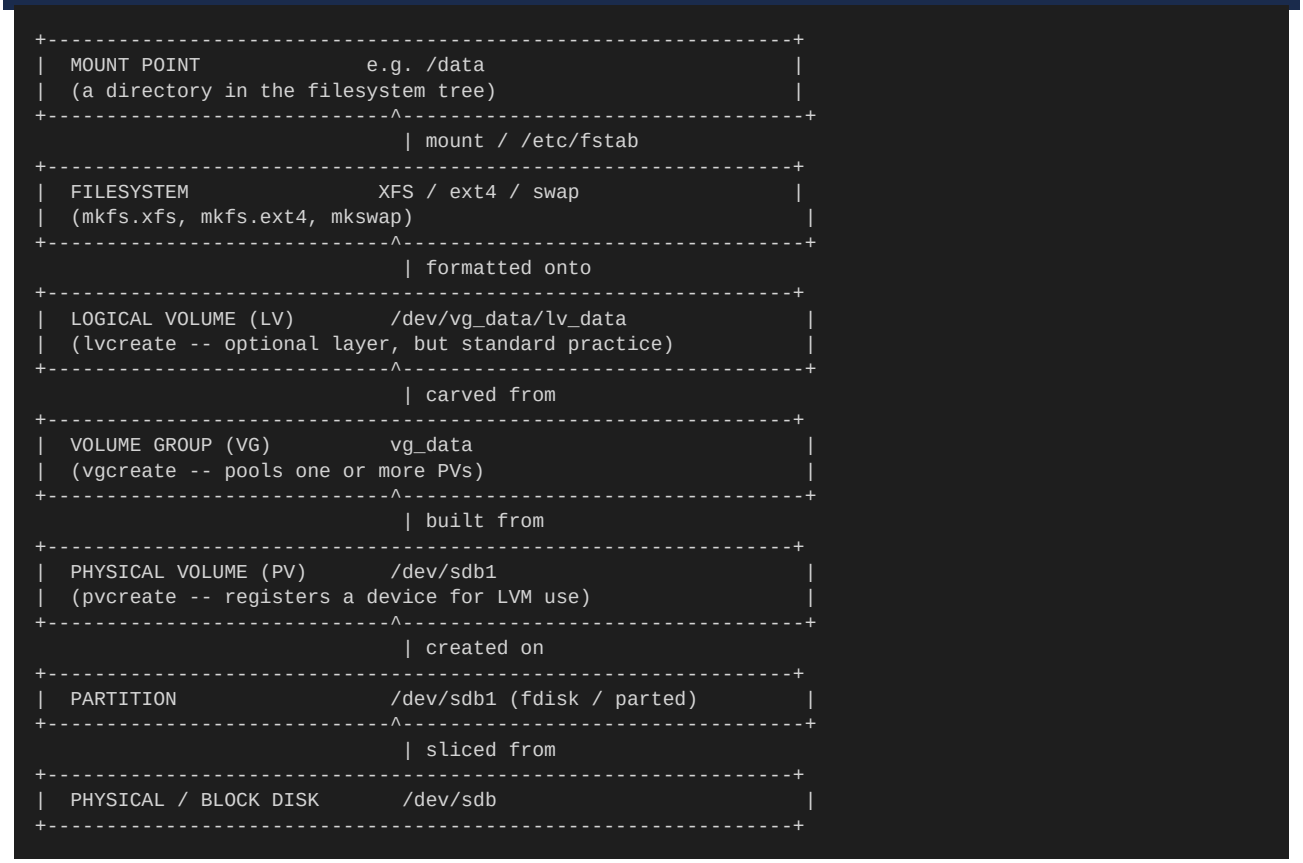
```
cp /etc/fstab /etc/fstab.bak # ALWAYS back up first
echo 'UUID=00000000-0000-0000-0000-000000000000 /broken xfs defaults 0 0' >> /etc/fstab
mkdir -p /broken
reboot
```

Recovery — at the emergency shell

```
# systemd drops you to a root shell (may ask for root password)
mount -o remount,rw / # ensure root is writable
vi /etc/fstab # delete/fix the bad line
mount -a # validate the fix before rebooting
systemctl default # OR just reboot again
```

Self-check: *Why does one bad line in fstab block the ENTIRE boot, even for unrelated mounts? (Hint: think about default mount options and the 'nofail' option — try repeating this lab adding* `,nofail` *to the bad line and see how boot behavior changes.)*

11. Quick Revision — The Whole Stack in One Picture



Swap branches off at the same level as a filesystem — it sits directly on a partition, LV, or file, activated with `swapon` instead of `mount`, and has no mount point.

End of Guide — practice each lab in a disposable VM/snapshot before touching production systems.